

GhostInTheDark

L3ak CTF 2025

Scenario - A removable drive was recovered from a compromised system. Files appear encrypted, and a strange ransom note is all that remains.

The payload? Gone.

The key? Vanished.

But traces linger in the shadows. Recover what was lost.

Password to open zip file is - L3akCTF

Solve

Looking in the zip

```
(kali@kali) - [~/Desktop/GhostInTheDark]
$ unzip -l Ghost.zip
Archive:  Ghost.zip
  Length      Date    Time    Name
-----
104857600    2025-06-26 22:50    GhostInTheDark.001
-----
104857600                                1 file
```

First to open the zip use password L3akCTF

Looking at the file we see a disk image.

```
(kali@kali) - [~/Desktop/GhostInTheDark]
$ file GhostInTheDark.001
GhostInTheDark.001: DOS/MBR boot sector MS-MBR Windows 7 english at offset 0x163 "Invalid partition table" at offset 0x17b "Error loading operating system" at offset 0x19a "Missing operating system", disk signature 0xf6a155; partition 1 : ID=0x7, start-CHS (0x0,2,3), end-CHS (0xb,254,63), startsector 128, 198656 sectors
```

We can use **mmls** which is a command-line utility within **The Sleuth Kit** to display the partition layout of the disk image.

```
(kali㉿kali) - [~/Desktop/GhostInTheDark]
$ mmls GhostInTheDark.001

DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors
```

	Slot	Start	End	Length	Description
000:	Meta	0000000000	0000000000	0000000001	Primary Table (#0)
001:	-----	0000000000	0000000127	0000000128	Unallocated
002:	000:000	0000000128	0000198783	0000198656	NTFS / exFAT (0x07)
003:	-----	0000198784	0000204799	0000006016	Unallocated

Next we will create a directory to mount the disk image.

```
(kali㉿kali) - [~/Desktop/GhostInTheDark]
$ sudo mkdir ./ghost
sudo mount -o ro,loop,offset=$((128 * 512)) GhostInTheDark.001 ./ghost
```

Mounting the drive we see some encrypted files and a ransom note.

```
(kali㉿kali) - [~/Desktop/GhostInTheDark]
$ ls ghost
flag.enc  payload.enc  ransom_note.txt  RIP_PuppyJaws.enc  'System Volume Information'  trip_itinerary.enc

(kali㉿kali) - [~/Desktop/GhostInTheDark]
$ cat ghost/ransom_note.txt
i didn't mean to encrypt them.
i was just trying to remember.

the key? maybe it's still somewhere in the dark.
the script? it was scared, so it disappeared too.

maybe you'll find me.
maybe you'll find yourself.

- vivi (or his ghost)
```

Using **fls** we can look at the image.

```
(kali㉿kali) - [~/Desktop/GhostInTheDark]
$ fls -r -o 128 GhostInTheDark.001

r/r 4-128-1:      $AttrDef
r/r 8-128-2:      $BadClus
r/r 8-128-1:      $BadClus:$Bad
r/r 6-128-4:      $Bitmap
r/r 7-128-1:      $Boot
d/d 11-144-4:     $Extend
+ d/d 29-144-2:   $Deleted
+ r/r 25-144-2:   $ObjId:$0
+ r/r 24-144-3:   $Quota:$0
+ r/r 24-144-2:   $Quota:$Q
+ r/r 26-144-2:   $Reparse:$R
+ d/d 27-144-2:   $RmMetadata
++ r/r 28-128-4:   $Repair
++ r/r 28-128-2:   $Repair:$Config
++ r/r 28-128-6:   $Repair:$Corrupt
++ r/r 28-128-8:   $Repair:$Verify
++ d/d 31-144-2:   $Txf
++ d/d 30-144-2:   $TxfLog
+++ r/r 32-128-2:  $Tops
```

Further down we see a `loader.ps1` the `-/r *` indicates the file was deleted but is able to be recovered.

```
r/r 42-128-1:     flag.enc
r/r 41-128-3:     payload.enc
r/r 38-128-1:     ransom_note.txt
r/r 43-128-3:     RIP_PuppyJaws.enc
d/d 36-144-1:     System Volume Information
+ r/r 37-128-1:   WPSettings.dat
r/r 39-128-1:     trip_itinerary.enc
-/r * 40-128-1:   loader.ps1
V/V 256:          $OrphanFiles
```

We can use **icat** to recover the file. Reading it we see the `payload.en c` was encrypted with AES and we have the key and iv.

```

(kali㉿kali) - [~/Desktop/GhostInTheDark]
$ icat -o 128 GhostInTheDark.001 40 > loader_recovered.ps1

(kali㉿kali) - [~/Desktop/GhostInTheDark]
$ cat loader_recovered.ps1
$key = [System.Text.Encoding]::UTF8.GetBytes("0123456789abcdef")
$iv = [System.Text.Encoding]::UTF8.GetBytes("abcdef9876543210")

$AES = New-Object System.Security.Cryptography.AesManaged
$AES.Key = $key
$AES.IV = $iv
$AES.Mode = "CBC"
$AES.Padding = "PKCS7"

$enc = Get-Content "L:\payload.enc" -Raw
$bytes = [System.Convert]::FromBase64String($enc)
$decryptor = $AES.CreateDecryptor()
$plaintext = $decryptor.TransformFinalBlock($bytes, 0, $bytes.Length)
$script = [System.Text.Encoding]::UTF8.GetString($plaintext)

Invoke-Expression $script

# Self-delete
Remove-Item $MyInvocation.MyCommand.Path

```

Taking that to **CyberChef** we find a similar script that is the actual ransomware. There is another Key and IV and also a flag that was taken from the C drive. Finally, the script encrypts any other file in the drive and deletes the original.

Recipe

From Base64

Alphabet: A-Za-z0-9+/=

☒ Remove non-alphabet chars

☐ Strict mode

AES Decrypt

Key: 0123456789abcd ... UTF8

IV: abcdef98765432 ... UTF8

Mode: CBC

Input: Raw

Output: Raw

total: 2
loaded: 2

File details

Name: payload.enc
Size: 2,074 bytes
Type: unknown
Loaded: 100%

Input

```

dM5Da2CieHn48b1fLpIeY+z9J5gcnZRMmIytGjHqYz+MpQpsKwCX7oTcGZk8gdSkLi8u0R09AJevztUknMK
7srHNf5w5BcwXN6RN8WP3MI7f1uw0YbgWbJwCmuA/
NWxzx5lEn0VWCrrgb4NN047lpoKHu4XbCF1vpQ2cdC18AHG6IH6rK2m3S94KSYdzjJDVFL5/
gi4SCrrC4B1apFf0vvG0JHYK1NPYEPgs1IQU5BZ7uZyVD4NUg349a/
eIaTwv8KY+LnqQ5k5mwgEBKAfaod98ybOnsWJ/3yQqh0LdjWX74W6L115TEkg/
+4AkPEiCxsDvmjeTPgK5MI5QexrQTt7Qrym4RMR6+rKZT7FpLmNVXgz84RqtjNN3Xu8lTtFMNQCMNVkb4NkE
lAvogw5LLXArYxe90i1h+0fn1FFcSim5hF7Xhe029oWlvx+j0SLVmZgjVHyxYch9Stt0/6Tb+GK5x87WMD4Z
bDAXcJ0ks7PPfP0UWzhPAMKeIwJx5H4LYEYkoxpvqwnAu5ESVkbVBu0Hsq4HMDdNwPCJ8o6qKg1INl505FM
oHqWlcvgJA+st9I1BaLqzLapsfa7t9S6n6ayF3HS1nA12KwXT533iS89AaYvpZkxivlyCLf82A2/2wEx5VoF
lZ22L5SA5lJsuUr+9h3w5zK0we/BvFb0Z2Ewn9hEWk8AmKSNJJfBn53uIM4Co4s+70q1lvZ/+r/
JKli1wGdLY3KgY545GScufIA0i7e0EY6mE6tUSjQ9FMqfssQEwCY/
fSSknzs4xnlmz+jicLbzyP4Y15XZvqVwXfYkEd013nmz5b6fF0XCE4mLw8hJyofMrHDZu0KVZSN+Iw+Esq6Q
31h0i0DztVtU0k44/30Xb6gnrLuMHIK1K2FtIEX1ryNseyxitkIEe7U48lXr/
10ZMGmtzFuuRoQbKfSWEZpcohaZsnamXjVmcU6HHI2M4Hj4WLKPQfz1DcRdyG0IzHn+YmE1N3tKrB3cnCH
hvJIDoA/l9s1TC134EXrkcWKS7gZF0gcA7j0jn+NcQ47vffDTAQu8/Y89boAe5JA9rQAFJYwG5P6tw0/B//
ZCD1RT6VPicFRLTYv07VklV4wL0iN3iU9HSr41J4tpImGhTTM1S1aR0V32qSHY362RvZfZ2Z2G7heKYhw19
0DXDn9rH8MtXeR99k15G7zGf0NfGdmfFbKwFS4n0Y4/

```

2674 2

Output

```

$key = [System.Text.Encoding]::UTF8.GetBytes("m4yb3w3d0nt3x1st")
$iv = [System.Text.Encoding]::UTF8.GetBytes("l1f31sf0rl1v1ng!")

$AES = New-Object System.Security.Cryptography.AesManaged
$AES.Key = $key
$AES.IV = $iv
$AES.Mode = "CBC"
$AES.Padding = "PKCS7"

# Load plaintext flag from C:\ (never written to L:\ in plaintext)
$flag = Get-Content "C:\Users\Blue\Desktop\StageRansomware\flag.txt" -Raw
$encryptor = $AES.CreateEncryptor()
$bytes = [System.Text.Encoding]::UTF8.GetBytes($flag)
$scipher = $encryptor.TransformFinalBlock($bytes, 0, $bytes.Length)
[System.IO.File]::WriteAllBytes("L:\flag.enc", $scipher)

# Encrypt other files stashed in D:\ (or L:\ if you're using L:\ now)

```

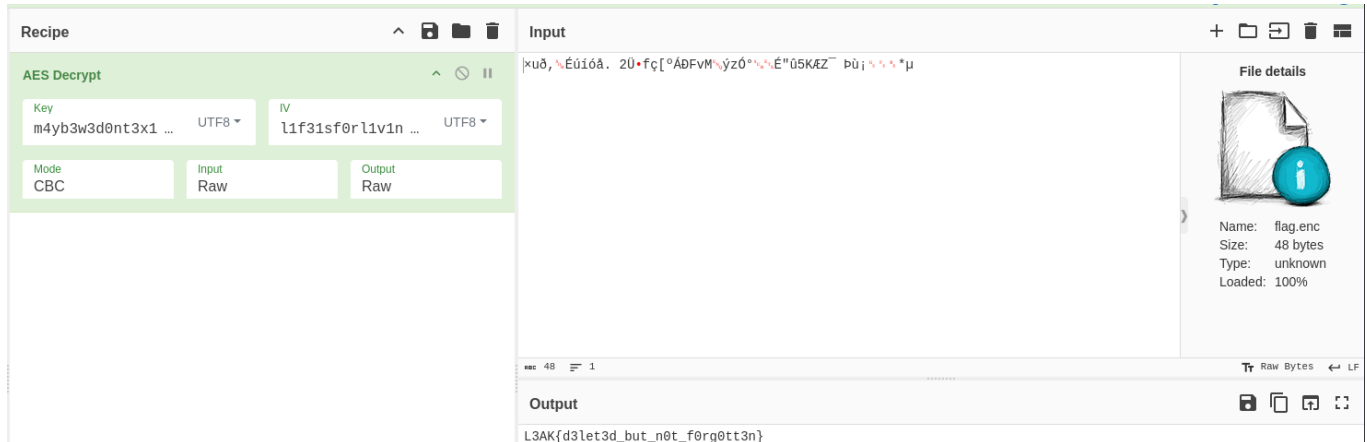
1545 47 340-395 (46 selected)

STEP

BAKE!

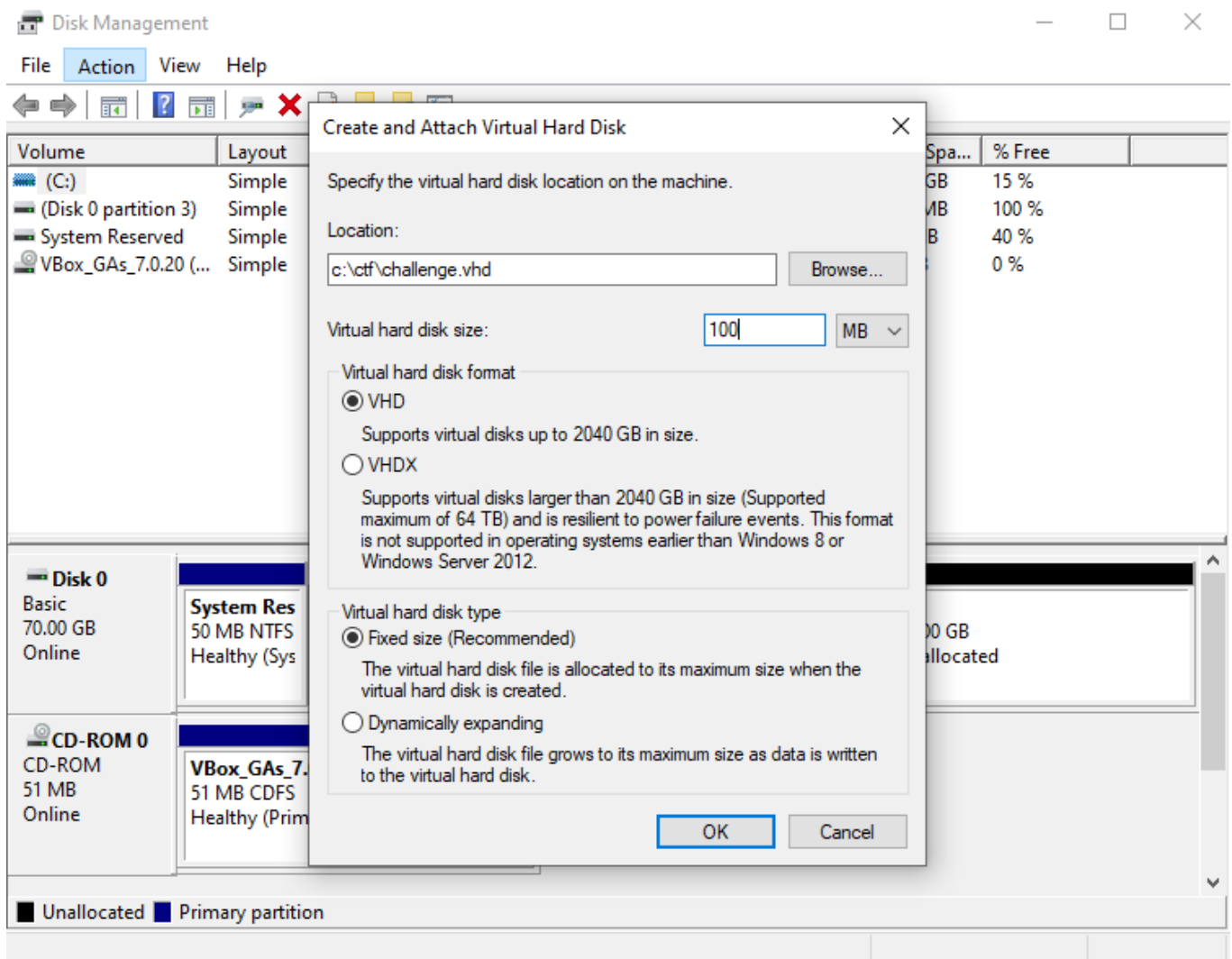
☒ Auto Bake

Using the same method as before, we can take the `flag.enc` to **CyberChef** with the new Key and IV and get the flag.

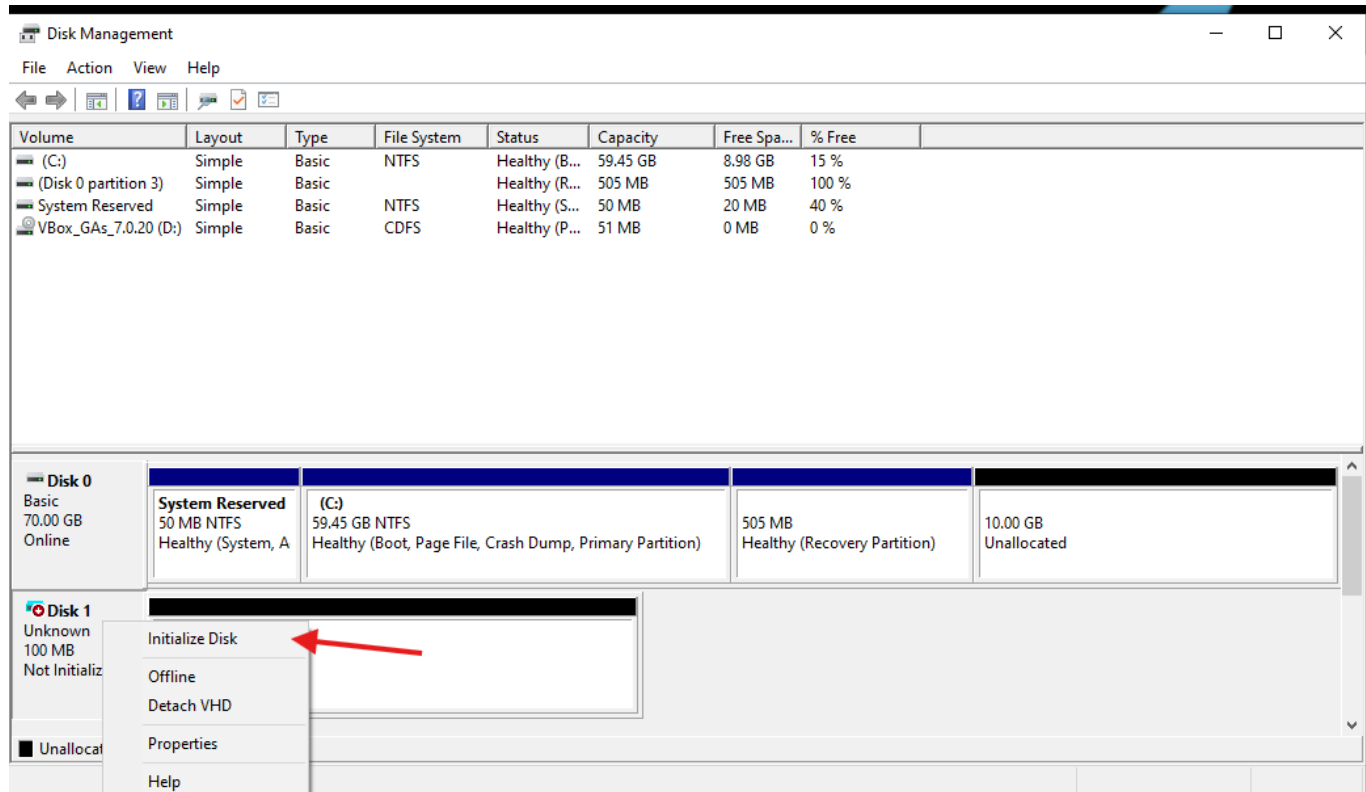


Behind the scenes - How the challenge was set up.

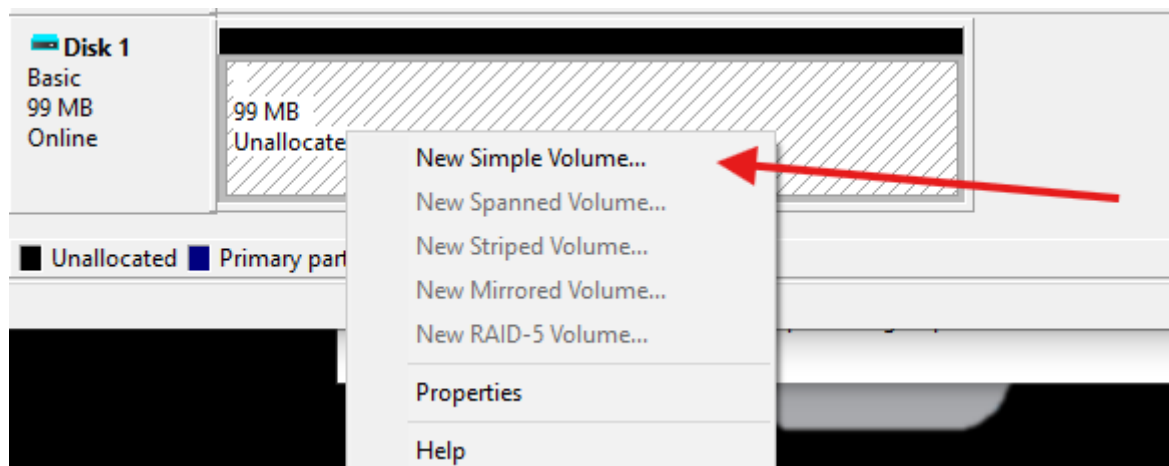
The first step is to create the Virtual Hard Disk.



Right Click the new Disk 1



Right click unallocated



Named Drive L

New Simple Volume Wizard ✕

Format Partition
To store data on this partition, you must format it first.

Choose whether you want to format this volume, and if so, what settings you want to use.

☐ Do not format this volume

☒ Format this volume with the following settings:

File system: NTFS ▼

Allocation unit size: Default ▼

Volume label: New Volume

☒ Perform a quick format

☐ Enable file and folder compression

< Back Next > Cancel

Now this is set up we can set up and deploy the malware.

To do so we will need a couple pieces. The key here is we don't want the plaintext flag to touch the drive. This could allow the players to use **strings** to recover it. Secondly, we don't want the payload to be saved on the drive in plaintext either, that would also L3ak the key and iv needed to decrypt `flag.enc`.